



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۴ آبان

فصل دوم: لایه *Application* اسلاید ۸۷ تا ۱۱۵ [۱]

کاربرد های *multi – media*

مثال هایی از آن *video streaming* و *CDN* است.
این کاربرد ها منابع زیاد و *bandwidth* زیادی نیاز دارند.
یک چالش مهم در این کاربرد ها، مقیاس پذیری و ناهمگونی *host*هاست. (یعنی *host*ها توانایی های مختلف دارند و *bandwidth*های مختلف)
برای برطرف کردن این چالش ها باید زیرساخت های توضیعی و در سطح کاربرد داشته باشیم.

video

هر تصویر آرایه ای از پیکسل و هر پیکسل تعدادی بیت است. ویدیو دنباله ای از این تصاویر است با نرخ $\#image/sec$ مشخص می شود.

انواع *coding*ها برای حذف *redundancy*:

افزونگی مکانی *spatial*: وقتی چند پیکسل کنار هم یک رنگ دارند.

افزونگی زمانی *temporal*: وقتی بین این فریم و فریم بعدی بیشتر پیکسل ها ثابت مانده، برای همین اختلاف فریم بعدی با این فریم را نگه میدارند.

این *coding*ها میتواند دو جور باشند:

$CBR = constant\ bit\ rate$: یعنی صدا و ویدیو با نرخ ثابتی کد میشوند.

$VBR = variable\ bit\ rate$: طبق همان افزونگی ها و تغییرات مکانی و زمانی، نرخ کدینگ تغییر می کند،

از انواع کدینگ های معروف: *MPEG1, MPEG2, MPEG4*

streaming stored video

یعنی قبلاً ویدیو روی سرور ذخیره شده. و مشتری ویدیو را تکه تکه دانلود میکند و پس از دانلود کردن چند تکه و همزمان با دریافت تکه های بعدی، آن را پخش میکند.

از آنجایی که *bandwidth* در *access – network in – house* و *core – network* و *video server* متفاوت است، و متغیر است. پس ممکن است با *congestion* در شبکه مواجه شویم. و در نتیجه *packet loss* داشته باشیم و کیفیت ویدیو ضعیف شود.

طبق اسلاید ۹۲، اگر تاخیر شبکه در هیچ لحظه ای متغیر نباشد، شکل موج دندان اره ای یکسانی مثل ارسال و دریافت آن داریم. بعد از اینکه چند فریم دریافت کرد، میتواند شروع به پخش کند.

continuous playout constraint

اینکه موج دندان اره ای مبدا و مقصد (مثل همانکه سرور دریافت کرده) یکسان باشند.

از آنجایی که شبکه *jitter* و تاخیر متغیر دارد، باید زمان پخش به گونه ای باشد که همیشه در بافر فریم باشد تا *continuous playout constraint* برقرار بماند با همان سرعت *original* پخش شود و اثرات نواسانات تاخیر، شکل موج اصلی و تامیمینگ اصلی ویدیو صدمه نزنند.

چالش های دیگر:

- تعاملات کاربر در مقصد با ویدیو: مثل عقب یا جلو یا ایست موقت، یا سریع به جلو برود یا پرش به مقاطع مختلف ویدیو داشته باشد.

- بسته های تلف شده: که آیا نیاز به باز ارسال دارند؟

طبق اسلاید ۹۴، موج پخش باید عین موج دریافت شده توسط سرور مبدا باشد. پس باید زمان پخش همیشه پایین تر از نمودار زمان دریافت فریم باشد

فاصله زمان دریافت اولین فریم تا لحظه پخش آن توسط *client play delay* مشخص می شود.

فاصله عمودی بین منحنی پخش (آبی) و منحنی دریافت (مشکی) میزان ویدیوی بافر شده در هر لحظه را نشان می دهد و باید این مقدار همواره بزرگ تر از صفر باشد (همیشه منحنی پخش از منحنی دریافت پایین تر باشد و از آن بالا نزد. و در بهترین حالت مماس شود)

اگر خیلی تاخیر بندازیم، خب همه ویدیو بافر می شود و ویدیو بدون مشکل پخش می شود. ولی اولاً بافر جا ندارد (نیاز به بافر بزرگ) و دوماً پخش لحظه ای و زنده نیست.

بهترین زمان پخش همان است که انقدر منحنی پخش را به منحنی دریافت نزدیک کنیم که مماس شود حداکثر، اما این امکان پذیر نیست چون ما از تاخیر های شبکه خبر نداریم.

DASH = dynamic adaptive streaming over http

این پروتکل از پروتکل HTTP کمک می‌گیرد. ویدیو ها به صورت *chunk* ها ذخیره می‌شود. هر چانک به چند نوع کد می‌شوند. چانک های با کیفیت بهتر و چانک های با کیفیت پایین تر. این چانک ها در *CDN* های مختلف ذخیره می‌شوند. اطلاعات چانک ها و اینکه کجا ذخیره شده به همراه *URL* آن سرور ها در فایل *manifest file* که در سرور اصلی است ، ذخیره می‌شود. در واقع مشتری اول *manifest file* را از سرور می‌گیرد. درواقع مشتری به صورت دوره ای پهنای باند را اندازه گیری میکند، چون متغیر است. درواقع با توجه به *bandwidth*، کیفیت و نرخ فایل هایی که دریافت میکند، تغییر میکند. پس مشتری تعیین میکند که چه زمانی و با چه کیفیتی طبق پهنای باند، و از کجا (کدام سرور) فایل را دریافت کند. اینکه از کدام سرور بگیرد طبق الگوریتم های مختلف میتواند رخ دهد. مثلا نزدیک ترین سرور یا سرور با *bandwidth* بالاتر. پس *video streaming* از کد کردن و پروتکل *DASH* و بافر کردن در مقصد، تشکیل شده.

CDN = content distribution network

برای توزیع ویدیو بین کاربران دو راه داریم
یا میتوان یک سرور بزرگ داشت (که *single point of failure* داریم و مقیاس پذیر نیست و ازدحام در سرور رخ میدهد. سرور از بعضی مشتریان دور است. اگر درخواست کاربران با فاصله زمانی از هم باشد، ممکن است چندین نسخه از یک ویدیو روی همان یک لینک ارسال شود که منجر به ازدحام می‌شود).
یا میتوان از *CDN* استفاده کرد. چندین نسخه از یک ویدیو روی چندین سرور یا سایت ذخیره می‌شوند. پیاده سازی های مختلفی دارد.

روش enterdeep

در شبکه دستیابی هر ناحیه جغرافیایی سرور *CDN* قرار می‌گیرد و نزدیک هر کاربر یه سرور *CDN* است.

روش bring home

سرور های *CDN* در *POP site* ها هستند و نه لزوما شبکه های دستیابی. پس سرور های کمتری دارند.

مشتریان فایل *manifest* را از سرور اصلی (مثلا نتفلیکس) می‌گیرد و درخواست خود را به *CDN* ارسال می‌کند.

(مثلا نزدیک ترین *CDN* اگه سرش شلوغ بود، یکی دیگه رو انتخاب میکنه و از اون ادامه ویدیو را میگیرد.) نکته اینکه قبلا نتفلیکس اون ویدیو را به *CDN* داده است دو نوع *video streaming* وجود دارد.

• *in network*: شبکه‌های دستیابی تحت مالکیت شرکت‌های ارائه دهنده آن هستند. کاربران باید عضو این شرکت‌ها شوند تا ویدیو را ببینند.

• *OTT = over – the – top*: از بستر اینترنت فعلی با *CDN*های *enter deep* استفاده میکند. تغییر زیادی در اینترنت لازم ندارند و از همان بستر *best – effort* و بدون ضمانت استفاده میکند. درواقع *CDN* یک لایه روی اینترنت ایجاد میکند و کاربران و *CDN*ها از اینترنت به عنوان یک سرویس استفاده میکنند.

چالش‌های *OTT*:

شبکه *best – effort* زیرین، ممکن است دچار ازدحام شود. حال در حالت ازدحام مشتری از کدام گره *CDN* دریافت کند و با چه نرخ؟ چه محتوایی و در چه گره *CDN* ذخیره شود؟ یکم از مثال گیج شدم. مثال نتفلیکس معقول تر بود. این که مانیفست یک فیلم رو از نتفلیکس میگیره و طبق آن مانیفست به *CDN* مربوطه وصل شده و با پروتکل *DASH* فیلم را میبیند.

توی مثال قبلی هم نکته این بود که اول کاربر با سرور اصلی ارتباط می‌گرفت بعد *DNS* محلی کاربر با *DNS* سرور اصلی و بعد با *DNS* سرور *CDN* ارتباط می‌گرفت و بعد که مانیفست را (از *CDN*) دریافت کرد (مراحل ۵ یا ۶)، کاربر مستقیم با سرور اصلی *CDN* ارتباط گرفته و ویدیو را میبیند. حتما بعدا اسلاید ۱۰۱ و ۱۰۲ رو نگاه کن.

برنامه نویسی *Socket*

سوکت، یک در است که میان پروسس کاربرد و پروتکل *transport* واقع شده است. سوکت‌های سرویس‌های لایه انتقال *UDP* و *TCP* متفاوت هستند.

UDP

قبل از ارسال داده بین مشتری و سرور، اتصال و *handshake* برقرار نمی‌شود و مشتری، شماره پورت و *IP* گیرنده را به تمام بسته‌ها اضافه می‌کند. و *OS* به طور خودکار (و نه کد کاربر *UDP*) پورت و *IP* فرستنده را به بسته‌ها اضافه می‌کند. پس گیرنده میتواند آدرس و پورت فرستنده را از بسته دریافتی استخراج کند. امکان اینکه بسته‌ها *Loss* شوند و خارج از ترتیب برسند وجود دارد.

این پروتکل، امکان انتقال تعدادی از بایت‌ها معروف به *datagram* را میدهد، اما این انتقال *unreliable* است.

با فیلد *SOCK_DGRAM* مشخص میکنیم که داریم از *UDP* و سوکت آن استفاده می‌کنیم. به مقدار $serverPort = 12000$ در اجرای برنامه نیاز داریم. حواست باشه. کد پایتون سمت کاربر و سرور برای *UDP* در اسلایدها آمده.

TCP

سرور باید اول در حال ران شدن باشد. کاربر باید با سرور از طریق سوکت *welcome* سرور تماس بگیرد. (در واقع سرور روی پورت *welcome socket* در حال گوش دادن است) برای تماس باید کاربر سوکت *tcp* ایجاد کند که در آن شماره پورت و *IP* سرور مقصد را مشخص میکند. اگر با سوکت *welcome* سرور تماس بگیرد، سرور با یک سوکت جدید غیر از سوکت *welcome*ش با کاربر صحبت می‌کند. (تا سوکت *welcome* آزاد شود تا کلاینت‌های دیگر هم بتوانند درخواست دهند).

این باعث می‌شود، که سرور با چندین مشتری صحبت کند و برای تمییز مشتریان از شماره پورت و *IP* آنها استفاده می‌کند.

درواقع یک سرویس انتقال *byte – stream* است (برعکس *UDP* که دیتاگرام انتقال میداد). که مطمئن است و ترتیب را رعایت می‌کند. انگار یک پایپ بین پروسس مبدا و مقصد وجود دارد.

پس در شروع کار سرور در حال اجرا و منتظر درخواست اتصال است. در پایان کار هم سرور اتصال را می‌بندد.

کد پایتون مربوط به *TCP* در کلاینت و سرور در اسلایدها آمده

با فیلد *SOCK_STREAM* مشخص میکند که دارد از سوکت *TCP* استفاده می‌کند.

مراجع

[1] “ch2-v8-slide-87-to-115,” <https://drive.google.com/drive/folders/>, 2022.